KINGS PORTAL GLOBAL

PRODUCT GUIDE

A client-facing overview of the website, member portal, mobile application, and the platform behind them.

One connected digital platform for church discovery, community engagement, and ministry operations.

Prepared for client review | 31 October 2026

What this guide covers

The public website; signed-in member and staff journeys; mobile app feature guide; role permissions; technology architecture; key integrations; operating assumptions; and a practical path from setup through launch. This document describes the current project as found in the supplied codebases and identifies items that require production configuration.

Reading the status language “Implemented” means represented in the current project code. “Configuration dependent” means it requires credentials, service setup, or deployment settings to operate fully. “Recommended behavior” describes the intended user-facing outcome and should be treated as a product acceptance expectation.

1. Executive overview

Kings Portal Global (KP Global) brings a church’s public presence and day-to-day digital services into one connected product. Visitors can find services, branches, ministries, events, sermons, devotionals, giving instructions, and contact routes. Members use a portal or mobile app for Bible reading, prayer, notes, notifications, and community updates. Authorized leaders manage branches, people, events, content, and ministry operations through role-scoped dashboards.

Product outcomes

- Make it easier for a newcomer to understand the church and take a next step.
- Give members a consistent place to engage with Scripture, sermons, events, prayer, and church communications.
- Reduce manual coordination for branch teams through shared records, role permissions, and operational tools.
- Keep public website and mobile data connected to the same Django services and church records.

Product at a glance

Audience	Primary value	Main entry points
Visitors	Discover the church and plan a visit	Website home, New Here, Locations, Events, Ministries
Members	Participate and stay informed	Website account/portal, mobile app
Workers	Coordinate ministry commitments and content	Worker dashboard, app Manage area
Branch administrators	Run their branch operations	Branch dashboard and scoped management tools
Super administrators	Configure and oversee the full organization	Global dashboard, website settings, app management tools

Delivery status snapshot

The codebase includes a working Django website and API, a React Native / Expo mobile client, and several connected user journeys. A final production release still depends on deployment, production email and push configuration, live service credentials, content population, and end-to-end acceptance across real devices. “Available in code” should not be read as “already published in app stores” or “live in production.”

2. How the product fits together

The website and mobile app are separate user interfaces that rely on the Django platform for shared accounts, records, content, permissions, and API services. The website renders pages on the server. The mobile app calls versioned JSON endpoints and keeps user authentication tokens in secure device storage.

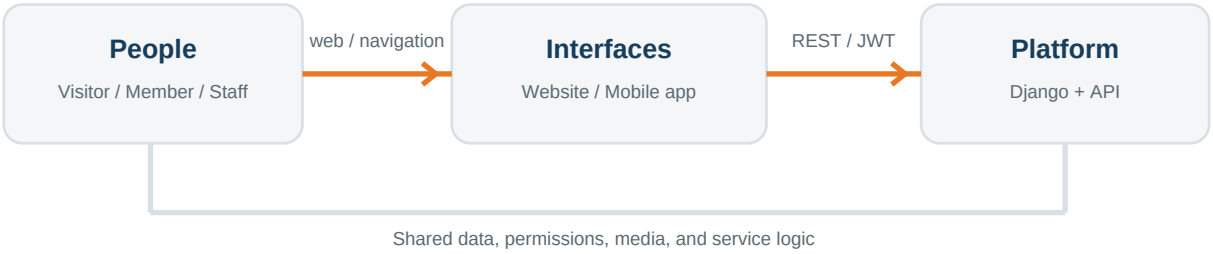

Figure 1. Shared platform view: both interfaces connect to the Django service layer and common data.

Core service journey

Step	What happens	User-visible result
1. Discover	A visitor browses public pages and content.	Church information is available without signing in.
2. Join	A visitor creates an account or signs in.	A member account opens the relevant personal experience.
3. Participate	Members read, watch, submit requests, and manage preferences.	Content and personal records remain associated with the account.
4. Coordinate	Staff create and update branch-scoped operational records.	Members see relevant events, announcements, and assignments.
5. Communicate	The platform can register devices and send notifications.	Push delivery works after credentials and provider setup.

A shared backend reduces duplication: the same branch, event, sermon, devotional, and announcement concepts support both web and mobile experiences.

3. Public website guide

The public website is the front door for people who are exploring KP Global, planning a visit, or looking for current church information. Pages use a consistent shared site layout and draw on managed records where the project supports them.

Page / feature	How a visitor uses it	Expected behavior
Home	Scan the welcome message, featured content, upcoming events, announcements, and calls to action.	Lead to clear next steps: attend, explore, watch, pray, or give. Content should be current and links should work on mobile and desktop.
About	Learn the church story, beliefs, leadership, and purpose.	Present concise, approved information and route visitors to branches or contact details.
New Here	Find practical first-visit information and a low-friction next step.	Answer what to expect, where to go, and how to connect; avoid requiring an account to enquire.

Page / feature	How a visitor uses it	Expected behavior
Locations	Browse branch locations and service information.	Show accurate addresses and contact/service details for each active branch.
Ministries	Browse ministry areas and open a ministry detail page.	Each detail should explain who it serves, what it does, and how to get involved.
Events	Browse upcoming events and open event details.	Show dates, times, location, description, and relevant branch; expired/cancelled events should be clearly handled.
Sermons	Browse the sermon archive and open individual messages.	YouTube content is retrieved server-side; pagination and cached fallback support the archive.
Live stream	See a live or upcoming stream indicator when applicable.	The public layout polls live status on a schedule and should stop polling outside relevant windows.
Devotionals	Read published devotional content.	Present title, date, Scripture, reflection, and any action prompt in a readable format.
Gallery	Browse church imagery and stories.	Use approved media, responsive presentation, and accessible image descriptions.
Prayer	Submit a prayer request through the public form.	Validate fields, confirm receipt, and apply the privacy expectations set by church leadership.
Praise	Share a praise report / testimony.	Show a clear submission outcome; moderation should govern public display.
Give	Read giving instructions and follow the configured giving link if supplied.	Current project displays instructions; it does not process card payments or reconcile transactions.
Contact / Subscribe	Send an enquiry or subscribe to updates.	Validate contact details and give a clear success or error state.

Recommended visitor journey

Home → New Here or Locations → Events / Ministries → Contact or prayer. Keep these routes visible in navigation and repeat the primary visit action on relevant pages. This makes the website useful as a guided journey, not only as a catalogue of pages.

4. Member and staff portal

After sign-in, the portal presents a role-specific dashboard. Permissions are enforced on the server: the page a user sees and the records the user can retrieve or change are both scoped by role and, for branch administrators, by assigned branch.

Role	Scope	Typical tasks
Member	Own profile and personal member services	View dashboard and notifications; update profile/password; apply to become a worker; track application status.
Worker	Own profile plus permitted ministry work	Use worker dashboard; review assignments/rotas; manage permitted updates or content; view notifications.
Branch administrator	Assigned branch records; global content read-only	Manage branch members/workers, events, announcements, notifications, and worker applications.
Super administrator	Organization-wide	Manage branches, roles, Bible versions, settings, content, giving records, prayer/praise queues, and platform-wide operations.

Operational areas

Area	What staff do	Expected behavior
Branches	Create and maintain branch information and administrators.	Branch-scoped operators cannot access another branch’s private records.
Members and workers	Find people, review profiles, update worker status, and maintain records.	Search and filter records; confirm consequential role or status changes.
Worker applications	Review requests, approve or decline, and track outcomes.	Applicants remain members until an administrator approves them; decisions should be reflected in the applicant’s status.
Events and rotas	Schedule events and publish service/team assignments.	Dates and times are clear; rota assignments are visible only to relevant users.
Announcements and notifications	Create updates for a branch or wider audience and deliver notices.	Audience/scope should be explicit before publishing or sending.
Sermons and devotionals	Manage devotional content; refresh sermon metadata where supported.	Sermon media is sourced from YouTube; devotional publication is controlled by the content team.
Prayer and praise	Review requests and reports, update status, and moderate public display.	Sensitive prayer information should be accessible only to authorized roles.
Giving records	Record and review gifts entered by staff.	This is manual record keeping; no online transaction or payment gateway is implemented.
Bible management	Upload, replace, and delete Bible JSON version files.	Validate file type, size, JSON content, and version naming; limit access to super administrators.
Stories and pastors	Manage public stories and leadership information.	Only approved content should be published; uploaded media depends on configured storage.
Notes	Create, edit, and delete portal notes.	Keep content ownership and access behavior clear for the relevant role.
Settings	Maintain church identity, links, contact information, and service settings.	Changes should be validated and reflected consistently on website/app surfaces.

Access model

Self-registration creates a Member. Administrators assign higher privileges. Branch administrators manage only their assigned branch records; a non-authorized attempt should be rejected or hidden. This provides a product-level boundary for member information and operational data.

Recommended management workflow

Create or update the underlying content → preview the audience and details → publish/save → verify the public or member view → revise or archive when stale. For private requests, review and update status without exposing internal notes publicly.

5. Mobile app guide

The mobile app is built with React Native and Expo Router. Its navigation is organized around Home, Sermons, Events, Profile, and a role-aware Manage area, with additional service screens for Bible, devotionals, notes, quiz, giving, prayer requests, announcements, praise, and account tools. The sections below explain the current product intent and how a user should use each area.

Feature	How to use it	What the feature should do
Onboarding	Open the app and move through the introductory slides; continue to sign-up or sign-in.	Explain the app's value and guide first-time users into a clear account or browse path.
Sign-up and verification	Enter account details, select a branch where requested, complete profile/preferences, and verify the emailed code.	Validate each step, show progress, provide resend/error states, and create the account with Member access. Email delivery requires production email configuration.
Sign-in and recovery	Sign in with email and password; use forgot password and verification/reset steps if needed.	Return a clear error for invalid credentials, securely refresh tokens, and return the user to an appropriate screen after recovery.
Home	Review welcome area, verse of the day, devotion preview, next service/event, announcements, quick links, reading progress, and role banner.	Surface timely actions and personalized items; tapping a card opens the matching feature.
Sermons	Browse sermon/video cards and select one to play.	Provide archive navigation, video playback, details, and live status when available. YouTube API configuration may be required.
Events	Browse upcoming events, switch date/category as available, and open event details.	Show accurate timing, branch/location, and event description with refresh and empty states.
Bible	Choose a version, book, and chapter; read verses; select/share a verse and navigate chapters.	Keep reading controls accessible, preserve selected version and location, and support locally available Bible text. Additional versions depend on published API files.
Devotionals	Open today's devotional or library item; read Scripture and reflection; mark progress/continue reading.	Show progress consistently and avoid losing a user's place during navigation.
Notes	Create a note, format text, save it, reopen/edit, or delete it.	Persist notes to the correct account when signed in; explain sync/offline behavior if connectivity is interrupted.
Quiz	Answer Bible quiz questions and review score/feedback.	Explain scoring and completion; questions are drawn from the bundled question set.
Prayer request	Enter and submit a prayer request from the app.	Confirm receipt and show the request's privacy/status expectations.
Praise report	Submit a praise report/testimony.	Confirm submission and allow moderation before public publication.
Offering / giving	Open the giving information or configured giving link.	Clearly distinguish external giving from in-app payment. No gateway is integrated in the current project.
Announcements	Read current church or branch updates.	Show items based on account access and relevant branch/global audience.

Feature	How to use it	What the feature should do
Notifications	Review inbox, mark one or all as read, and manage preferences.	Unread count and read status should stay synchronized; push delivery requires device/provider setup.
Profile and account	Edit personal details, change password, update notification settings, or sign out.	Validate changes, protect account data, and provide successful-save confirmation.
Worker application	Complete the application form and check its status later.	Prevent duplicate submissions where applicable and show pending/approved/declined state.
Manage	Authorized staff use the management dashboard/tools for their role.	Only show permitted management links; server-side authorization remains authoritative.

Suggested first-time member path

Install and open → complete onboarding → register and verify email → choose branch/profile preferences → land on Home → explore Events, Bible, or Sermons → enable notifications when prompted and desired. Each step should explain why information or permission is requested.

Connectivity and persistence

The app uses network status support and local storage libraries. API-backed records require a reachable service. The Bible bundle is available locally; content such as events, announcements, notifications, and account data may require connectivity. The interface should show a usable error/empty state and let users retry instead of silently dropping actions.

6. Technology and integrations

Layer	Technology in project	Product role
Mobile client	Expo SDK 57, React Native 0.86, React 19.2, Expo Router, JavaScript/JSX	Cross-platform iOS/Android app and web-compatible route setup. Note: repository instructions asked us to consult Expo v56 docs; the actual app dependency is SDK 57, so use matching v57 docs for implementation work.
Website / backend	Django 5.2, Python, Django templates, HTML/CSS/JavaScript	Server-rendered public site, accounts, member/role dashboards, and business logic.
Mobile API	Django REST Framework, SimpleJWT, versioned `/api/v1/` routes	JSON services for login, member data, events, announcements, sermons, Bible versions, notifications, and management.
Data	PostgreSQL configured for production; SQLite option for local/demo use	Stores accounts, branches, church records, and operational content.
Media	Cloudinary storage configuration; local/static asset support	Stores uploaded story and other media in deployed environments. Requires account credentials.
Sermons / livestream	YouTube Data API service with caching and fallback	Loads sermon archive and live/upcoming stream status. Requires channel/API configuration and is subject to quota.
Device capabilities	SecureStore, notifications, sharing, image picker, video, local storage, network status	Supports secure token storage, push registration, verse sharing, media selection, video viewing, and device-aware UX.
Supporting services	Email/SMTP, Redis cache (optional), Expo push services	Verification/reset messages; shared cache in production; push-token registration and delivery workflows.

Architecture diagram

The application is a two-client product over a shared service layer. Website requests use Django views and session authentication. The mobile app uses REST endpoints and JWT access/refresh tokens. External media and delivery services are integrations, not substitutes for the core record system.

Important configuration and release requirements

- Provide a production HTTPS API URL to the app and production allowed-host, CORS, and CSRF configuration to Django.
- Configure SMTP and verify email-code and password-reset delivery outside development. The default email backend writes codes to the server console.
- Configure YouTube API/channel settings and monitor quota, stale-cache behavior, and stream visibility.
- Configure Expo push credentials/provider setup, test device registration and delivery, and establish opt-in language.
- Set secrets, PostgreSQL, backups, media storage credentials, and shared Redis cache where appropriate; disable development cleartext settings in production.
- Build and validate native iOS and Android release artifacts, app-store metadata, signing, privacy declarations, and support contact details.

SDK documentation note

The project-level AGENTS.md requires consulting the versioned Expo v56 documentation before writing code. That reference was reviewed. The app manifest currently declares Expo `~57.0.30` with React Native `0.86.3`; the SDK 56 reference describes a different React Native target. This guide documents the code as configured and does not change it. Future code changes should check the exact docs for the installed SDK version and resolve this repo instruction/version mismatch.

7. Feature flows and expected outcomes

These short flows are useful for client walkthroughs and acceptance discussions. They describe a complete, user-visible journey rather than only a screen inventory.

Journey	User action	System response / expected completion
Visitor plans a visit	Open New Here → select a location → review service/event details → contact the church.	Current location and contact details are shown; enquiry/visit action has a visible confirmation route.
Member joins	Register → verify email → finish profile → arrive at Home.	Account is created with Member role; user sees success and role-appropriate content.
Member submits prayer	Open Prayer Request → enter request → submit.	Validation runs; request is stored; user sees a receipt confirmation; only authorized staff can review private content.
Staff publishes event	Open Manage → Events → create event → choose branch/date/details → save.	Event appears in staff list and, when published/confirmed, in public/app event discovery.
Leader reviews worker application	Open Applications → inspect details → approve or decline.	Decision is saved; applicant status changes; approved user receives worker access according to policy.
Member watches sermon	Open Sermons → browse/select message → play video.	Sermon metadata and player are shown; cache or clear retry state handles upstream availability.
Member reads Scripture	Open Bible → select translation/book/chapter → read/select/share verse.	Text loads, navigation is clear, and share action creates a readable citation/verse card.
Staff records giving	Open Giving → create/edit a record.	Record is saved for internal tracking; user is not charged and no payment is processed.

Success states matter

For forms and account actions, success should be explicit: indicate what was submitted or changed and what happens next. For all remote content, show loading, empty, offline, and error states. For potentially sensitive actions such as approving workers or publishing prayer/praise content, clearly display the scope and effect before saving.

8. Operational ownership and content lifecycle

The product depends on reliable ownership of content and configuration. A simple operating rhythm protects trust: keep service information current, review requests promptly, and use role boundaries to control who can change or see records.

Content / operation	Suggested owner	Review rhythm
Service, branch, and contact details	Church operations lead	At least quarterly and whenever details change
Events and rotas	Branch administrator / ministry coordinator	Weekly review; archive expired items
Announcements and push notifications	Communications lead / authorized branch staff	Before every send; remove or correct stale items
Sermons and livestream setup	Media lead	Check channel configuration and upcoming stream schedule regularly

Content / operation	Suggested owner	Review rhythm
Devotionals and Bible versions	Content lead / super administrator	Before publication; confirm Scripture references and files
Prayer requests and praise reports	Pastoral team / designated reviewer	Frequent review with appropriate confidentiality
Accounts, roles, and branch assignments	Super administrator	On onboarding, role change, and staff departure
Backups, secrets, email, storage, and deployments	Technical operator	Ongoing monitoring and documented recovery checks

Support guidance

For a user problem, collect the platform (website/iOS/Android), app version or browser, account role, time of occurrence, and the screen or action involved. Never request passwords or verification codes. For content corrections, route the request to the staff owner of that content area.

9. Delivery plan from setup to launch

The projects are structured to support the following delivery sequence. It converts the existing implementation into a stable, supportable client release.

Phase	Work	Exit outcome
1. Confirm scope	Approve feature inventory, role expectations, content owners, privacy rules, and launch geography.	Agreed product scope and acceptance checklist.
2. Configure services	Set production secrets, database, storage, domain/HTTPS, email, YouTube, cache, and push credentials.	Production-like environment with integrations connected.
3. Populate and validate	Load branches, service details, ministries, leaders, events, devotional content, Bible versions, and giving instructions.	Accurate, approved content across website and app.
4. Verify journeys	Walk through visitor, member, worker, branch admin, and super admin flows on desktop and actual iOS/Android devices.	Issues triaged; key user journeys accepted.
5. Release	Build/sign apps, prepare store listing and privacy materials, deploy website/API, verify backups and monitoring.	Public website and mobile release available to intended users.
6. Operate and improve	Review analytics/support themes, content freshness, notification delivery, permissions, and release feedback.	Owned maintenance and a prioritized improvement backlog.

Suggested acceptance checks

- A visitor can find a branch, upcoming service/event, contact route, and giving instructions without an account.
- A new member can register, verify, sign in, recover access, and update profile/password.
- A branch administrator cannot read or modify another branch’s restricted records.
- A worker application progresses through pending and a recorded decision.
- A sermon can load from YouTube configuration and the app provides a helpful state if the service is unavailable.
- A user can submit prayer/praise content and staff can moderate it with the expected privacy.
- Notifications can be registered and delivered in release builds, with consent and settings behavior explained.

- Giving instructions and manual records do not imply an in-app payment unless a gateway is added and separately accepted.

10. Scope notes and product decisions

This document is a product guide based on the supplied mobile and Django project workspaces, not a claim that every external service is already configured or every deployment is live. The implementation currently contains mock or locally configured portions in some service layers; before a client launch, confirm each journey against the production API and real credentials.

Topic	Current project position	Decision / follow-up
Website	Public page templates and role dashboards are present.	Confirm final public copy, domain, accessibility, browser support, and which sections are active at launch.
Mobile app	Expo app includes onboarding, auth, tabs, and service/manage screens.	Confirm release build state, app store publication, supported OS range, device QA, and whether each screen is API-backed.
Authentication	Django supports accounts and JWT endpoints; app has multi-step sign-up and recovery screens.	Configure production email, verification policy, secure token lifecycle, and account support process.
Giving	Giving instructions plus manual gift records.	Choose an external giving provider if online payments are required.
Push notifications	Device token registration and send workflows exist in project.	Configure delivery credentials and define opt-in, audiences, quiet hours, and consent language.
Live stream / sermons	YouTube-backed archive/live detection with cache behavior.	Confirm channel, API quota, scheduled stream metadata, and fallback content.
Data protection	Role and branch checks exist in server-side code.	Agree retention, consent, prayer-request visibility, media rights, backups, and incident/support contacts.
Analytics	No formal analytics product is described in the current code documentation.	If needed, decide what to measure and obtain appropriate privacy/legal review before adding tracking.

Recommended client walkthrough

Start with the public website and demonstrate the visitor path. Then show a member sign-in and the Home, Bible, Events, Sermons, Prayer, and Profile experiences. Finish with a staff dashboard: select a branch, show scoped event/member operations, and explain how super administrator access differs. Keep the giving and push-configuration caveats explicit during the walkthrough.

Appendix A. Feature index

A compact directory for client reference. Detailed descriptions appear in the relevant sections above.

Channel	Included feature groups
Public website	Home; About; New Here; Locations; Ministries and details; Events and details; Sermons and details; live status; devotionals; gallery; prayer; praise; give; contact; subscribe.
Member portal	Member dashboard; profile; password change/reset; notifications; worker application and status; member-facing notes/services as available.

Channel	Included feature groups
Staff portal	Branch/member/worker management; applications; ministries; events; rotas; pastors; announcements; devotionals; sermon tools; prayer/praise moderation; giving records; stories; Bible versions; notifications/push; settings; notes.
Mobile app	Onboarding; registration/login/recovery; Home; Sermons; Events; Profile; Manage; Bible and versions; devotionals/progress; notes; quiz; prayer; praise; offering; announcements; notifications; worker application; account settings.
Platform services	Django templates; REST API; JWT; shared data and permissions; PostgreSQL/SQLite configuration; media storage; YouTube integration; email; optional Redis; push registration/delivery.

Appendix B. Reference points

Repository sources reviewed: mobile `README.md`, `package.json`, `app.json`, `src/app/` route tree, API client/endpoints and service modules; Django `README.md`, settings, root/app URL configurations, website/dashboard templates, API routes, and project requirements. Expo documentation reference read per the repository AGENTS.md: Expo SDK v56 documentation. The configured package version in the app is Expo SDK 57, as noted above.

Document version 1.0 | Prepared 31 October 2026